Unit II

2. Sorting and Linked list:

2.1 Selection sort
2.2 Bubble sort
2.3 Insertion sort
2.4 Introduction to Linked list
2.5 Representation of Linked list in memory
2.6 Searching a linked list
2.7 Memory allocation, Garbage collection
2.8 Insertion and deletion in linked list

2.1 Selection sort:

Definition:

Selection Sort is a simple comparison-based sorting algorithm that repeatedly selects the smallest (or largest) element from the unsorted portion of the array and places it in its correct position.

Steps of Selection Sort:

1. **Start with the first element:** Assume it is the smallest element.
2. Compare it with all other elements in the unsorted part of the array.
3. Find the smallest element and swap it with the first element of the unsorted portion.
4. Repeat the process for the remaining unsorted part of the array until the entire array is sorted.

Example:

Sort the array `[64, 25, 12, 22, 11]` in ascending order.

Steps:

1. Initial array: `[64, 25, 12, 22, 11]`
   Find the smallest element (`11`) and swap with the first element.
   Result: `[11, 25, 12, 22, 64]`

2. Next pass: `[11, 25, 12, 22, 64]`
   Find the smallest element in the unsorted part (`12`) and swap with the second element.
   Result: `[11, 12, 25, 22, 64]`

3. Next pass: `[11, 12, 25, 22, 64]`
   Find the smallest element (`22`) and swap with the third element.
   Result: `[11, 12, 22, 25, 64]`

4. Final pass: `[11, 12, 22, 25, 64]`
   The array is now sorted.

Time Complexity:

Best Case: $O(n^2)$

Worst Case: $O(n^2)$
Space Complexity: O(1) (In-place sorting algorithm)

Advantages:
Simple to implement.
Does not require additional memory.

Disadvantages:
Inefficient for large datasets due to $O(n^2)$ complexity.

2.2 Bubble sort:

Definition:
Bubble Sort is a simple comparison-based sorting algorithm where adjacent elements are repeatedly compared and swapped if they are in the wrong order. This process continues until the entire array is sorted.

Steps of Bubble Sort:
1. Start with the first element: Compare it with the next element.
2. Swap the elements if they are in the wrong order (e.g., for ascending order, the larger element should be moved right).
3. Repeat the comparison for each adjacent pair in the array. After the first pass, the largest element "bubbles" to the end of the array.
4. Repeat the process for the remaining unsorted portion of the array.
5. Stop when no swaps are required in a pass (the array is sorted).

Example:
Sort the array `[5, 3, 8, 6, 2]` in ascending order.

Steps:
1. Initial array: `[5, 3, 8, 6, 2]`
   Compare `5` and `3` → Swap → `[3, 5, 8, 6, 2]`
   Compare `5` and `8` → No swap → `[3, 5, 8, 6, 2]`
   Compare `8` and `6` → Swap → `[3, 5, 6, 8, 2]`
   Compare `8` and `2` → Swap → `[3, 5, 6, 2, 8]`

2. Second pass: `[3, 5, 6, 2, 8]`
   Compare `3` and `5` → No swap → `[3, 5, 6, 2, 8]`
   Compare `5` and `6` → No swap → `[3, 5, 6, 2, 8]`
   Compare `6` and `2` → Swap → `[3, 5, 2, 6, 8]`

3. Third pass: `[3, 5, 2, 6, 8]`
   Compare `3` and `5` → No swap → `[3, 5, 2, 6, 8]`
   Compare `5` and `2` → Swap → `[3, 2, 5, 6, 8]`

4. Fourth pass: `[3, 2, 5, 6, 8]`
   Compare `3` and `2` → Swap → `[2, 3, 5, 6, 8]`

Final array: `[2, 3, 5, 6, 8]`

Time Complexity:

Best Case: O(n) (when the array is already sorted).

Worst Case: $O(n^2)$ (when the array is sorted in reverse order).

Space Complexity: O(1) (In-place sorting).

Advantages:

Simple to implement.

Detects sorted arrays quickly with the optimization of stopping when no swaps are made.

Disadvantages:

Inefficient for large datasets due to $O(n^2)$ complexity.

2.3 Insertion sort:

Definition:

Insertion Sort is a simple sorting algorithm that builds the final sorted array one element at a time by repeatedly picking an element from the unsorted portion and inserting it into its correct position in the sorted portion.

Steps of Insertion Sort:

1. Start with the second element (consider the first element as already sorted).
2. Compare the current element with the elements in the sorted portion of the array.
3. Shift all elements in the sorted portion that are greater than the current element to the right.
4. Insert the current element into its correct position.
5. Repeat the process for all remaining elements in the unsorted portion of the array.

Example:

Sort the array `[7, 4, 5, 2]` in ascending order.

Steps:

1. Initial array: `[7, 4, 5, 2]`
   First pass: Compare `4` with `7`.
   Shift `7` to the right → `[7, 7, 5, 2]`.
   Insert `4` → `[4, 7, 5, 2]`.

2. Second pass: `[4, 7, 5, 2]`
   Compare `5` with `7`.
   Shift `7` to the right → `[4, 7, 7, 2]`.
   Insert `5` → `[4, 5, 7, 2]`.

3. Third pass: `[4, 5, 7, 2]`
   Compare `2` with `7`, `5`, and `4`.
   Shift `7`, `5`, and `4` to the right → `[4, 5, 7, 7]`, `[4, 5, 5, 7]`, `[4, 4, 5, 7]`.
   Insert `2` → `[2, 4, 5, 7]`.

Final array: `[2, 4, 5, 7]`

Time Complexity:

Best Case: O(n) (when the array is already sorted).

Worst Case: $O(n^2)$ (when the array is sorted in reverse order).

Space Complexity: O(1) (In-place sorting).

Advantages:

Simple and easy to implement.

Efficient for small datasets or nearly sorted arrays.

Disadvantages:

Inefficient for large datasets due to $O(n^2)$ complexity.

.

2.4 Introduction to Linked list:

Definition:

A **Linked List** is a linear data structure where elements (called nodes) are connected using pointers. Each node consists of:

1. **Data:**
   The value stored in the node.

2. **Pointer/Link:**
   A reference to the next node in the sequence.

Unlike arrays, linked lists do not require contiguous memory allocation.

Types of Linked Lists:

1. **Singly Linked List:**
   Each node points to the next node, forming a unidirectional chain.

2. **Doubly Linked List:**
   Each node contains two pointers, one pointing to the next node and the other to the previous node.

3. **Circular Linked List:**
   The last node points back to the first node, forming a circular chain.

Features of Linked List:

1. **Dynamic Size:**
   Linked lists can grow or shrink during runtime, unlike arrays with a fixed size.

2. **Efficient Insertion/Deletion:**
   Adding or removing elements is easier and faster as no shifting is required.

3. **Sequential Access:**
   Unlike arrays, elements in a linked list must be accessed sequentially starting from the head.

Structure of a Node (Example in C):

```
struct Node {
    int data;
    struct Node* next; // Pointer to the next node
};
```

Advantages:

1. **Efficient Memory Usage:**
   No need for pre-allocated memory like arrays.

2. **Easy Insertion/Deletion:**
   Operations are O(1) if the pointer to the node is provided.

Disadvantages:

1. **Sequential Access:**
   Accessing an element requires traversal, making it slower (O(n)).

2. **Extra Memory:**
   Requires extra memory for storing pointers.

Use Cases:

Implementation of stacks, queues, and graphs.
Managing dynamic memory in operating systems.

2.5 Representation of Linked list in memory:

Definition:

In a **linked list**, each element (node) is stored in separate memory locations. These nodes are linked together using pointers, making it a non-contiguous data structure.

Node Structure:

Each node in a linked list typically consists of two parts:

1. **Data Field:**
   Stores the actual data (e.g., an integer, string, etc.).

2. **Pointer/Link Field:**
   Stores the address of the next node in the sequence.

Example Node Structure in C:

```
struct Node {
    int data;        // Data field
    struct Node* next; // Pointer to the next node
};
```

Memory Representation:

1. **Non-Contiguous Memory:**

Unlike arrays, where elements are stored in contiguous memory blocks, the nodes of a linked list are scattered across memory.

Each node is linked using a pointer, which stores the memory address of the next node.

2. Head Pointer:

The head pointer is a special pointer that stores the address of the first node in the linked list.

If the list is empty, the head pointer is set to `NULL`.

3. Last Node:

The pointer of the last node in a singly linked list points to `NULL`, indicating the end of the list.

Example:

Consider a linked list with elements: `10 → 20 → 30`.

Node 1: Data = 10, Pointer = Address of Node 2

Node 2: Data = 20, Pointer = Address of Node 3

Node 3: Data = 30, Pointer = `NULL`

Memory Representation:

Address	Data	Pointer
0X100	10	0x104
0X104	20	0x108
0X108	30	NULL

The `Head` pointer stores `0x100`, the address of the first node.

Advantages of Linked List Representation:

1. Dynamic Memory Usage:

Nodes are created only when required.

2. No Fixed Size:

Can grow or shrink as needed.

3. Efficient Insertion/Deletion:

Involves updating pointers rather than shifting elements.

Disadvantages of Linked List Representation:

1. Memory Overhead:

Extra memory is needed for storing pointers.

2. Sequential Access:

Requires traversal to access elements.

2.6 Searching a linked list:

Definition:

Searching a linked list involves traversing through its nodes to find a specific value or key. The search can either return the position of the node containing the value or confirm that the value is not present in the list.

Types of Search in Linked Lists:

1. Linear Search:

The most common method for searching a linked list.

Each node is sequentially checked starting from the head until the target value is found or the list ends.

2. Binary Search (not commonly used)**:**

Applicable only to **sorted linked lists** and usually requires conversion to an array due to lack of random access.

Steps for Linear Search:

1. Initialize a pointer (`current`) to the head of the linked list.

2. Compare the value in the `current` node with the target value.
   If they match, return the node or its position.
   Otherwise, move the pointer to the next node.

3. Repeat until the end of the list is reached (`current == NULL`).

4. If the value is not found, return an indication (e.g., -1 or "not found").

Example:

Consider the linked list: `10 → 20 → 30 → 40`.

Task: Search for the value `30`.

1. Start at the head (`10`): Compare `10` with `30` → No match.
2. Move to the next node (`20`): Compare `20` with `30` → No match.
3. Move to the next node (`30`): Compare `30` with `30` → Match found.
4. Return the position or pointer to the node.

Time Complexity:

Best Case:

O(1) (if the value is in the first node).

Worst Case:

O(n) (if the value is in the last node or not present).

Space Complexity:

O(1) (no additional memory required).

Advantages of Searching in Linked Lists:

Efficient for small or unsorted lists where insertion/deletion is frequent.

Disadvantages:

Slower compared to arrays due to the lack of random access (O(n) for all cases).

Requires traversal from the head for every search operation.

2.7 Memory allocation, Garbage collection:

Memory Allocation:

Definition:

Memory allocation refers to the process of assigning memory to a program or variable during its execution. It ensures that the program has the necessary memory space to store and manipulate data.

Types of Memory Allocation:

1. Static Memory Allocation:

Memory is allocated at compile-time.

The size of variables or arrays must be predefined.

Cannot be resized during runtime.

Example: `int arr[10];`

2. Dynamic Memory Allocation:

Memory is allocated during runtime.

The size of the memory block can be adjusted as needed.

Requires explicit allocation and deallocation by the programmer.

Example in C:

```
int* ptr = (int*)malloc(sizeof(int)); // Allocates memory dynamically
free(ptr); // Frees the memory
```

Advantages:

Dynamic memory allocation allows flexibility for data structures like linked lists, queues, and stacks.

Efficient utilization of memory for large and varying data.

Garbage Collection:

Definition:

Garbage collection is the process of automatically reclaiming memory that is no longer in use by a program. It prevents memory leaks by identifying and deallocating unused memory.

How It Works:

1. Mark-and-Sweep Algorithm:

Marks all active objects in memory.

Sweeps and removes unmarked objects (garbage).

2. Reference Counting:

Each object keeps a count of references to it.

When the count drops to zero, the memory is deallocated.

3. Generational Garbage Collection:

Objects are divided into generations (young, old).

Frequent garbage collection is performed on younger generations.

Key Features:

Automated in languages like Java and Python.

In manual systems like C/C++, the programmer must use `free()` or `delete` to deallocate memory.

Comparison:

Feature	Static Allocation	Dynamic Allocation	Garbage Collection
When	Compile-time	Runtime	Runtime
Flexibility	Fixed Size	Resizable	Automatic
Manual/Automatic	Manual	Manual	Automatic in managed languages

Advantages of Garbage Collection:

Reduces programmer errors (e.g., memory leaks, dangling pointers).

Simplifies memory management in high-level languages.

Disadvantages:

Can cause performance overhead during execution.

Requires sophisticated algorithms for efficient implementation.

2.8 Insertion and deletion in linked list:

Insertion in a Linked List:

Insertion involves adding a new node to a linked list. Depending on the position, there are three types of insertion:

1. Insertion at the Beginning:

Create a new node.

Set the new node's `next` pointer to the current `head`.

Update the `head` pointer to point to the new node.

Time Complexity: O(1).

2. Insertion at the End:

Create a new node.

Traverse the list to find the last node.

Set the last node's `next` pointer to the new node.

Set the new node's `next` pointer to `NULL`.

Time Complexity: O(n).

3. Insertion at a Specific Position:

Traverse the list to find the node just before the desired position.

Create a new node and update its `next` pointer to point to the next node.

Update the previous node's `next` pointer to point to the new node.

Time Complexity: O(n).

Example of Insertion:

Linked List: `10 → 20 → 30`

Task: Insert `15` at position 2.

1. Create a new node with data `15`.
2. Traverse to the node at position 1 (`10`).
3. Set the new node's `next` to point to the node at position 2 (`20`).
4. Update the `next` pointer of node `10` to point to the new node.

Result: `10 → 15 → 20 → 30`.

Deletion in a Linked List:

Deletion involves removing a node from the linked list. Depending on the position, there are three types of deletion:

1. Deletion from the Beginning:

Update the `head` pointer to point to the second node.

Free the memory of the first node.

Time Complexity: O(1).

2. Deletion from the End:

Traverse the list to find the second last node.

Set the second last node's `next` pointer to `NULL`.

Free the memory of the last node.

Time Complexity: O(n).

3. Deletion from a Specific Position:

Traverse the list to find the node just before the node to be deleted.

Update its `next` pointer to skip the node to be deleted.

Free the memory of the deleted node.

Time Complexity: O(n).

Example of Deletion:

Linked List: `10 → 20 → 30 → 40`

Task: Delete node with value `30`.

1. Traverse to the node with value `20`.
2. Update its `next` pointer to skip the node `30` and point to `40`.
3. Free the memory of node `30`.

Result: `10 → 20 → 40`.